A Complete Front-End Functional Verification Environment for a SIMT GPGPU

Samuel Moussa
Graduation Project
Email: kemetsemiconductors@gmail.com

***Abstract*—We report a front-end functional verification environment for the Vortex RISC-V SIMT GPGPU, built and closed by a single engineer. The environment provides two independent checking layers—byte-exact end-state equivalence against a golden reference model and per-instruction lockstep in the writeback domain—and, unusually, *proves both layers can fail* through committed fault-injection tests that must report an error on every regression. Coverage reaches 94.7% total and 98.1% of covergroup bins at the primary configuration and 94.6%/95.8% at two clusters, with 50 of 50 runs passing at both. Every structural coverage exclusion is generated per-configuration from register-transfer parameters with a cited justification, and a blocking merge-time invariant rejects any exclusion that changes a *covered* bin count; that check found two waivers which had been silently deleting real coverage. The campaign found real design defects, including an instruction-set specification deviation and an IEEE-754 accuracy error. We state the environment's maturity by axis rather than claiming closure: checker strength and coverage integrity are strong, whereas stimulus volume, error and exception handling, and formal methods remain open. We argue that reporting this boundary explicitly is what separates a verification result from a coverage number.**

***Index Terms*—functional verification, UVM, SIMT, GPGPU, RISC-V, coverage closure, non-vacuity, golden reference model**

I. Introduction

A verification environment is easy to mistake for a verification result. A bench that compiles, runs a regression, and reports high coverage may still be checking nothing: if the checker cannot fail, its silence is not evidence. This paper reports an environment built to make that failure mode detectable, and states plainly where the resulting campaign is strong and where it is not.

The design under verification is Vortex, an open-source RISC-V SIMT GPGPU. It is a demanding target for a small team: a full graphics-processor hierarchy of clusters, sockets, cores, warps and threads, with a configurable cache hierarchy, custom SIMT control instructions outside the base instruction set, and a memory model that is explicitly weakly coherent.

A. Contributions

1) A complete Universal Verification Methodology environment for a SIMT GPGPU, with role-inverted agents (the design is the bus master; the environment supplies responders), a golden model integrated through the direct programming interface, and a passive observability layer bound into the register-transfer hierarchy so that it scales to any topology without enumerating paths.
2) **Non-vacuity as a standing gate.** Both checking layers have committed negative tests that inject a fault and *must* report an error. They are re-run after every change, so a refactor that silently disables checking fails the regression rather than improving it.
3) **Coverage-exclusion integrity.** Exclusions are generated per-configuration from elaborated design parameters, each with a cited justification, and a merge-time invariant blocks any exclusion that changes a covered-bin count. We report two defects this found, one of which had been active for weeks.
4) **An honest maturity assessment** by axis, including the axes where the campaign is weak, and a categorized roadmap separating remaining front-end work from back-end sign-off work.

B. What this paper does not claim

We do not claim the design is verified, nor that it has been stressed. Neither is supportable. The randomized generator is seed-controlled and reproducible, but the committed regression runs one seed per profile, which is directed testing in a random costume. The bus responder always returns a successful response, so no error-recovery path is exercised. There is no architectural exception or interrupt stimulus, no formal property verification, and no X-propagation or randomized-reset analysis. These are stated here, in the introduction, because a limitation disclosed only in a final section invites the reader to discover it as a flaw rather than read it as scope.

II. Environment Architecture

A. Role inversion

In a conventional bench the environment drives transactions into a subordinate design. Here the relationship is inverted: the design is a program-executing bus master, so the environment's memory agents are *responders*. Stimulus is therefore selected primarily by choosing which program to execute, not by randomizing bus transactions. This distinction matters for how randomization is claimed: the random axis is the generated program, not the transaction stream.

Four agents are active—host control, device-configuration registers, and two memory responders (bus-protocol and native)—alongside a passive status agent and a set of register-transfer probes.

B. Golden model integration

A functional emulator that ships with the design serves as the reference, integrated through the direct programming interface. It steps instruction-by-instruction, models full SIMT architectural state, and returns retirement records suitable for lockstep comparison.

Two consequences are load-bearing and are reported rather than hidden. First, the reference is *not independent*: it was co-designed with the design under verification, so a shared misunderstanding is invisible to every check. Second, the reference and the design execute the *same binary*, so a defect in the stimulus itself is common-mode and cancels—a green run with zero mismatches is compatible with the program having verified nothing. We return to this in Section III-C.

C. Passive observability by binding

Probes are bound into the register-transfer hierarchy rather than connected by hierarchical path. A probe bound to a module definition is instantiated wherever that module is, so an environment written for one core becomes correct for sixteen with no edit. Probes are strictly read-only and never gate a verdict; they exist to make coverage attributable, not to check.

One case required more care. The device-configuration bus is write-only, so no front-door read exists and "did this configuration write reach the register?" had never been checked. A bound peek-only probe supplies the read side, enabling a register model with a real read-back check. The probe never writes: a backdoor write would desynchronize the golden model, which is fed from the bus monitor.

III. Checking, and Proving the Checkers

A. Two layers

The primary checker compares final memory state byte-exactly against the reference model. It is bidirectional: a forward pass compares every location the design wrote, and a reverse pass compares result-region locations the design *failed* to write. The reverse pass exists because a dropped store is invisible to the forward pass—there is no transaction to compare.

The secondary checker performs per-instruction lockstep in the writeback domain, comparing program counter, destination register and value at retirement.

B. Non-vacuity as a standing gate

Both layers have committed negative tests. One injects a wrong value into a result location; the other drops a store. Both must report an error, and both are re-run after every change to the environment. When the scoreboard was later rebuilt onto a single data source, the negative tests were confirmed still failing at the same address—which is what makes the refactor trustworthy rather than merely green.

We regard this as the most important property of the environment. A checker that has never been observed to fail is indistinguishable from an absent one, and most published verification results do not report evidence that their checkers can fail at all.

C. The silent-failure class

Because both models execute the same binary, a stimulus defect cancels. This is not hypothetical. Two instances were found in this project.

In the first, a compute kernel derived a tile geometry from a compile-time template parameter rather than from the runtime device configuration. The kernel was scaled to a larger topology, the geometry silently did not follow, and both models computed the same wrong answer and agreed. The end-state compare passed and coverage filled.

In the second, found while preparing the cache-hierarchy experiment reported in Section V, a kernel's phase-enable flags were discarded by the build system: a command-line variable overrode the makefile assignment that added them. The kernel compiled with every phase disabled, executed, compared byte-exactly against the reference, and passed—having exercised nothing. The defect was invisible in the simulation log and visible only by comparing the two compiler invocations directly.

The general lesson is that *only what differs between the two models is validated*; everything shared requires an independent assertion. We now treat the build path as part of the verified surface.

IV. Coverage Methodology

A. Exclusions as generated artifacts

Structural coverage exclusions are not maintained by hand. A generator emits them per-configuration from elaborated design parameters, so a waiver that is legitimate at one topology is not applied at another where the same logic is reachable. Each carries a citation to the register-transfer source that makes it structurally dead.

Two examples illustrate the discipline. The instruction cache is instantiated with its write port disabled, so its entire write-data path is elaborated but undrivable; this accounts for 26% of the design's toggle gap from a single subtree, and is waived with the instantiation cited. A positive control confirms the diagnosis: the corresponding response-data path toggles fully on all bits. By contrast, the upper bits of a 44-bit retirement identifier are *not* waived, because reaching them requires 2^{18} or more instructions—infeasible is not the same as structurally dead, and only the latter justifies removal from the denominator.

B. A blocking invariant on the merge

Exclusions are applied in two stages separated by justification class. Third-party intellectual property is excluded because it is not the design under verification, and legitimately removes covered bins. Structural exclusions must remove *only* unreachable bins. The merge therefore asserts a hits-invariant: after structural exclusions, no covered-bin count may change. Violation fails the merge.

This check found two real defects on its first execution. One was a waiver written the same day. The other had been silently discarding a covered condition term for weeks. The root cause in both cases is a tool property worth recording: focused

TABLE I
BANKED COVERAGE (PER CONFIGURATION; NEVER BLENDED)

Metric	1CL/1C/4W/4T	2CL/2C/4W/4T
Covergroup bins (raw)	370/377 = 98.1%	989/1032 = 95.8%
Covergroup (weighted)	99.8%	99.5%
Statement	98.1%	98.3%
Branch	95.1%	95.7%
Condition	90.4%	88.8%
Toggle	82.8%	80.5%
Assertion	96.9%	98.9%
Directive	100.0%	100.0%
Total	**94.7%**	**94.6%**
Runs passing	50/50	50/50
Coverage instances	2 256	8 275

expression coverage cannot waive a single input term of a condition—waiving one term discards the whole expression, including its covered rows.

C. What the total metric means

The tool's total coverage is the unweighted mean of seven categories, each contributing equally regardless of bin count. The lowest category is therefore the largest lever, independent of its size. We report the per-category breakdown rather than the total alone, because the total can be moved by a category with five bins as easily as by one with 400,000.

V. RESULTS

Table I reports the two banked configurations. Banks are never blended across configurations: instance counts and signal widths differ, so a merged database is not meaningful.

A. Directed stimulus for structural targets

Where a coverage gap resisted the existing suite, kernels were written against the specific missing terms and verified against those terms *before* entering the regression. One produced genuine internal back-pressure and covered thirteen of twenty-four missing condition terms at the primary configuration; another created a second concurrent memory stream by executing a large resident instruction footprint interleaved with data traffic.

B. A negative result on memory pressure

An intuition worth publishing because it is wrong: increasing memory pressure does not increase contention coverage. Two independent experiments enlarged a kernel's working set by a factor of sixteen. One went from thirteen covered condition terms to eleven, losing a target outright; the other went from four to zero.

The mechanism is that the terms in question require two requests live at one port in the same cycle. A working set that exceeds cache capacity sends nearly every access to main memory, every thread blocks, and issue stops. Requests still arrive, but spread thin—each drained long before the next. Total traffic rises while *concurrent* traffic falls. Contention is produced by issue density, not by miss volume. Both kernels carry this reasoning in their source, because the well-intentioned edit that would undo it looks like an optimization.

C. Exercising the shared cache hierarchy

The optional second- and third-level caches are instantiated in bypass form unless explicitly enabled, and no regression had ever enabled them: the suite had no path from its configuration to the compile. After supplying that path, a reuse-based kernel—touch a working set sized to a level, then re-read it—covered the hit path of all four shared-cache instances for the first time, with 13 000 to 15 000 hits each, while comparing 196 868 memory words byte-exactly against the reference with zero errors. This is the largest end-state comparison in the suite by a factor of six.

That configuration was banked separately (51 runs, all passing, 93.2% total). It is lower than the two-cluster bank without the shared levels, and the reason is instructive rather than disappointing: enabling two cache levels adds a large amount of new control logic—arbiters, flush state machines, miss-status handling—while exactly one kernel of fifty-one targets it. Condition coverage there is 79.3%. The honest summary is that the shared hierarchy was *reached and validated, not thoroughly exercised*: both halves of that sentence matter.

D. Defects found

The campaign found defects in both the design and the reference model, maintained in an evidence-cited register with explicit dispositions. The design-side findings include a deviation from the instruction-set specification in indirect-jump target computation and an IEEE-754 accuracy error of one unit in the last place in hardware square root. Several findings are observability limits or micro-architectural quirks rather than defects, and are recorded as such; a register that only ever records bugs is not being maintained honestly.

E. Provenance of the design under verification

The design is an open-source register-transfer model pinned at a specific revision, *with eighteen locally modified files*. Most changes are simulator-compatibility fixes made during bring-up. We disclose this because a verification result is a statement about a specific artifact, and "upstream, unmodified" would not describe what we ran.

One of those modifications turned out to matter, and its history is instructive. A library counter shipped with two assertions checking for overflow and underflow. They fired during bring-up and were guarded off—rewritten to skip whenever their inputs were unknown—so that the environment could run at all. That guard remained for months. On restoring the original assertion and root-causing why it fired, we found a genuine defect: the design distributes reset through a relay module that *registers* reset in a flip-flop which nothing resets. Its output is therefore unknown from time zero until the first clock edge, so every module behind a relay observes an unknown reset for one cycle; a reset-conditional then takes its non-reset branch and any assertion inside evaluates on unknown operands.

TABLE II
FRONT-END VERIFICATION MATURITY, SELF-ASSESSED

Axis	Status
Environment architecture	Strong
Checker strength / non-vacuity	Strong; above typical practice
Coverage integrity & waiver audit	Strong
Defect-discovery record	Strong
Reference-model quality	Good, but *not independent*
Directed stimulus	Good
Configuration breadth	Weak (three points)
Coverage-model provenance	Moderate; not specification-traced
Random stimulus volume	**Weak** (one seed per profile)
Error / exception verification	**Absent**
Formal property verification	**Absent**
X-propagation / reset randomization	**Absent**

We replaced the relay with an asynchronous-assert, synchronous-deassert synchronizer. The counter assertions then pass with no guard at all—twelve firings become zero—and the local modification to that counter was retired in favor of the unmodified upstream file. The defect is present in the current upstream release.

Two consequences deserve emphasis. First, **the assertion had been correct**: what was silenced was a real property of the design, not a testbench nuisance. A checker that is switched off to make a bench run is a checker whose findings are lost, and the cost here was months of an unobserved unknown-value window. Second, **fixing the defect reduced a coverage number**. Branch coverage fell from 95.1% to 94.5%: during the unknown-reset cycle, modules behind a relay had been executing their normal-operation paths, and those executions were *counted as covered branches*. With reset correct they no longer occur. Part of the previously reported branch coverage had been obtained from a state that cannot legitimately arise—which no coverage metric can reveal about itself. We report the lower figure as the correct one. (Two variables changed between those measurements, so the delta is indicative rather than isolated.)

VI. Maturity by Axis

We assess the campaign by axis rather than reporting a single readiness figure, because the axes are not substitutable: no amount of checker strength compensates for absent error-path stimulus.

Table II is deliberately unflattering in its lower half. The environment—the part that took the engineering—is substantially complete. The campaign that runs on it is not, and the deficit is concentrated in stimulus rather than infrastructure. That is an encouraging shape, because stimulus volume is bought with machine time once the generator is seed-controlled, which it now is.

A second structural limit deserves emphasis. The reference model is not independent of the design. An independent scalar cross-check was performed against a third model and agreed on all 11 076 base-instruction retirements, but that model has no SIMT semantics and cannot execute the design's custom control instructions. The SIMT axis therefore has no independent reference and will not get one from that direction. This is a ceiling, not an unfinished task.

VII. Toward Tape-Out

The two lists below are separated because they support different claims. Front-end items strengthen the claims in this paper; back-end items are prerequisites for a different claim entirely—that the design is manufacturable and will work in silicon. Nothing here speaks to the latter.

A. Remaining front-end work

1) **Stimulus volume.** Scale the seeded generator from one seed per profile to hundreds. Highest return in this list; costs machine time, not engineering.
2) **Error injection** on the bus responder (error and decode responses), opening the untested error-recovery axis.
3) **Exception and interrupt stimulus**, designed around the observation that this design terminates by deasserting a busy signal rather than by executing a breakpoint—the assumption that blocked earlier attempts.
4) **Formal property verification** on arbitration-heavy control where simulation is weakest and the state space is small.
5) **X-propagation and randomized reset**, to reach uninitialized-state behavior that a two-state flow cannot see.
6) **Configuration-matrix breadth** beyond three sampled points.
7) **Specification-traced coverage model**, so closure measures the specification rather than the model.

B. Back-end and ASIC sign-off work

1) Gate-level simulation, then with back-annotated timing.
2) Static timing analysis across process, voltage and temperature corners.
3) Design-for-test: scan, automatic test-pattern generation and fault-coverage sign-off; memory self-test.
4) Clock- and reset-domain-crossing analysis, structural and functional.
5) Lint and structural sign-off against a synthesis rule set.
6) Low-power verification: power intent, retention and isolation.
7) Physical closure: place and route, extraction, signal and power integrity.
8) Equivalence checking across source, netlist and post-layout netlist.
9) Post-silicon bring-up, characterization, and a path from silicon failures back into this regression.

VIII. Conclusion

We reported a front-end functional verification environment for a SIMT GPGPU in which both checking layers are proven able to fail, every structural coverage exclusion is generated from design parameters with a citation, and a blocking merge-time invariant prevents a waiver from consuming real coverage. Coverage closed at 94.7% total and 98.1% of covergroup

bins at the primary configuration, with all runs passing at both configurations, and the campaign found real defects in the design.

The environment is substantially complete; the campaign is not, and we have said where. Our broader argument is that this distinction is the substance of a verification result. A coverage number is a measurement of a model, and it is only as meaningful as the evidence that the checkers could have failed, that the waivers removed nothing real, and that the stimulus exercised what it claims. Where we could not supply that evidence, we have marked the axis open rather than closing it by assertion.